

Jira and Git Project Workflow

The most common workflow in modern software development teams is known as AGILE/SCRUM. This document aims to provide a high level overview of common practices with AGILE.

Sprints

Sprint is the term used to describe a set amount of time given to complete a set of goals. Sprints commonly last two weeks. Before the start of a sprint, Sprint Planning, Story Grooming and Sizing need to take place.

Sprint Planning, Story Grooming and Sizing

Before the start of a new sprint, a planning stage is required, where a Project Manager will break down requirements and goals for the coming sprint. The engineering team is often present at these meetings, and together, the team will break down the requirements into manageable sections, commonly known as Stories. Each story is commonly associated with a parent Epic.

Each story should have Acceptance Criteria (AC) listed in that story, as a guide for completing said story. After the AC has been determined by the team for a specific story, the team now needs to size that story.

Each organization and each team may size their stories differently. A common approach is to follow the 3-factor system: **Complexity, Uncertainty, Effort**. A number will be designated to each of the three factors listed, and an average of those numbers will highlight how many story points a story is worth, as well as give an estimation on how long a story may take to be completed. At the end of every sprint, each team member will be evaluated based on how many story points they were able to complete. Once sizing has been determined, priority will be given to each story (this is dependent on many factors, such as deadline).

Ex. Bug Fix - Broken Unit Test For An API

- **Uncertainty** would be low, as we have an existing unit test already in place (1)
- **Complexity** may be medium, as there may be edge cases unaccounted for, false positives, etc (4)
- **Effort** may also mirror Complexity (4)

So we can size this story as being worth 3pts.

Sample Story

Add epic / BGP-5

Registration page and Login Page Frontend and Backend

Attach

Create subtask

Link issue

Description

- AC1: user should be able to register(unique password, username,email)
- AC2: user should be able to login
- AC3: use localStorage/sessionStorage/cookie to make sure after login, whenever user refresh the page, the user should still be logged in.
- AC4:it should be able to send an actual email message to a valid email address.

Story Points

4

Activity

Show: All Comments History Work log

Newest first

EL

Add a comment...

Pro tip: press **M** to comment

To Do

Actions

Pinned fields

Click on the next to a field label to start pinning.

Details

Assignee

Zichan Yang (Anna)

Assign to me

Reporter

Zichan Yang (Anna)

Releases

Labels

None

Start date

None

Story point estimate

5

Team

NEW

None

Sprint

BGP

Priority

High

Sample Jira Board

Projects / HR-Group-Project

HGP Sprint 1

Deliver Node.js Backend and React Client

3 days remaining

Complete sprint

GROUP BY

None

Insights

View settings

TO DO 4

Registration page

FRONTEND(REACT)

HGP-5

3

HL

Onboarding Application page

FRONTEND(REACT)

HGP-6

4

HL

Personal Information page

FRONTEND(REACT)

HGP-8

3

JL

Visa Status Management page

FRONTEND(REACT)

HGP-9

3

JL

IN PROGRESS 4

Implement Validation

BACKEND

HGP-35

1

JY

Login page

FRONTEND(REACT)

HGP-4

3

S

Housing page

FRONTEND(REACT)

HGP-10

4

S

Implement Employee related API

BACKEND

HGP-36

11

JY

IN REVIEW 3

Create mongodb schemas for User, Report, Housing, RegisterToken

BACKEND

HGP-32

1

JY

Navigation bar

FRONTEND(REACT)

HGP-7

2

S

Implement token generation functionality

BACKEND

HGP-34

1

JY

DONE 4

UI Design(React)

FRONTEND(REACT)

HGP-14

UI Design(Angular)

FRONTEND(ANGULAR)

HGP-16

Data Model Design

BACKEND

HGP-15

API Endpoint Design

BACKEND

HGP-17

HGP-1 / HGP-5

Registration page

To Do

Actions

Description

Add a description...

Child issues

Order by

0% Done

HGP-13

Signup UI Form

1

TO DO

HGP-18

Signup function...

2

TO DO

Confluence pages

Product requirem...

TRY TEMPL...

Updated just now

Pinned fields

Click on the next to a field label to start pinning.

EL

Add a comment...

Pro tip: press **M** to comment

Git Workflow

Different teams have different workflows. On a high level, workflow will be some variation of the following:

Branching

When assigned a new feature, checkout a new local Git branch

```
Unset  
git checkout -b name-feature
```

The local master branch will act as your single source of truth. In the event something goes really wrong in your feature branch, you can always checkout a new branch from master. Depending on how often code is updated, ensure that you occasionally perform a “git pull” to ensure your local master branch is up to date with the remote repo’s master branch.

Pull Requests (PR)

Once your feature is completed and unit tests have been performed and documented, a Pull Request should be generated on Github. Before generating a new PR, ensure that your current feature branch’s head is up to date with the remote master branch’s head to avoid merging older code or running into merge conflicts. The Rebasing Strategy is most commonly used.

1. Checkout your local master branch
2. Perform a “git pull” to ensure your local master branch is synced with the remote master branch
3. Checkout your feature branch
4. Perform a “git rebase” to rebase the head of your current feature branch to align with your now updated local master branch
5. Commit and push

```
Unset  
git checkout master  
git pull  
git checkout feature-branch  
git rebase master  
git commit  
git push origin feature-branch
```

Once your code has been pushed to your remote feature branch, generate a PR. Different teams will have different conventions for how to title their PRs. No matter what the practice, it is beneficial to include the corresponding Jira story/ticket title and issue number for easier reference and tracking during development, future references, or if any issues/bugs arise. Here is a suggested format:

Jira/Epic - Issue Number [Scope of Work] Jira Story Title

Samples:

HGP-13 [Front-end Dev] Personal Info Page

HGP-10 [Back-end Dev] Personal Info API

HGP-12 [Back-end Bug Fix] Personal Info Update Error

Add a description of how this PR is addressing the ACs in a story, any notes regarding changes, or any other information that may be deemed important or relevant to the Jira story.

Sample PRs

The screenshot shows a GitHub Pull Request (PR) titled "SCRUM-30 Update Github repo readme.md and setup eslint #7". The PR is merged and was created 14 minutes ago by user ChuqiZeng. The interface includes a header with the PR title and a "Code" button. Below the header, there are tabs for "Conversation" (0), "Commits" (1), "Checks" (0), and "Files changed" (17). The main content area is divided into sections: "Description", "Changes", "Purpose", and "Testing". The "Description" section contains a paragraph about the PR's purpose. The "Changes" section lists three items: 1. ESLint Configuration (adding .eslintrc.cjs files), 2. README.md Updates (Root Directory) (expanding the README.md with GitHub Workflow Guidelines and ESLint setup), and 3. README.md Updates (employee-client-react Folder) (updating the README.md in the employee-client-react folder). The "Purpose" section lists three goals: ensuring consistency, providing clear guidelines, and streamlining code review. The "Testing" section is currently empty. On the right side, there are sections for "Reviewers" (No reviews), "Assignees" (ChuqiZeng), "Labels" (None yet), "Projects" (None yet), "Milestone" (No milestone), "Development" (Successfully merging this pull request may close these issues), "Notifications" (Subscribe button), and "1 participant" (ChuqiZeng).

SCRUM-30 Update Github repo readme.md and setup eslint #7

Merged ChuqiZeng merged 1 commit into dev from dev-chuqi 14 minutes ago

Conversation 0 Commits 1 Checks 0 Files changed 17 +2,845 -121

ChuqiZeng commented 15 minutes ago

Description

This pull request introduces ESLint configuration to both our server-side and client-side (employee-client-react) codebases. It also includes updates to the README.md files, detailing our GitHub workflow guidelines, ESLint setup, and coding standards.

Changes

- ESLint Configuration:
 - Added .eslintrc.cjs to server folder for backend linting rules.
 - Added .eslintrc.cjs to employee-client-react folder for React-specific linting rules.
- README.md Updates (Root Directory):
 - Expanded the README.md to include a new section on GitHub Workflow Guidelines. This section outlines the steps and standards for creating branches, submitting pull requests, and reviewing code.
 - Added an ESLint Guidelines section to help team members set up ESLint in their local development environments. This section includes instructions on installing dependencies, configuring editors (VS Code), and running ESLint.
- README.md Updates (employee-client-react Folder):
 - Updated README.md in the employee-client-react folder.
 - This README.md focuses specifically on naming rules for React components, hooks, and other relevant entities in our react-client-side codebase. It serves as a reference for maintaining consistency and clarity in our code.

Purpose

The primary goals of these changes are to:

- Ensure consistency and enhance code quality across both server and client-side code.
- Provide clear guidelines for team members on how to adhere to our coding standards.
- Streamline our code review and GitHub workflow processes.

Testing

Reviewers: No reviews

Assignees: ChuqiZeng

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues.

None yet

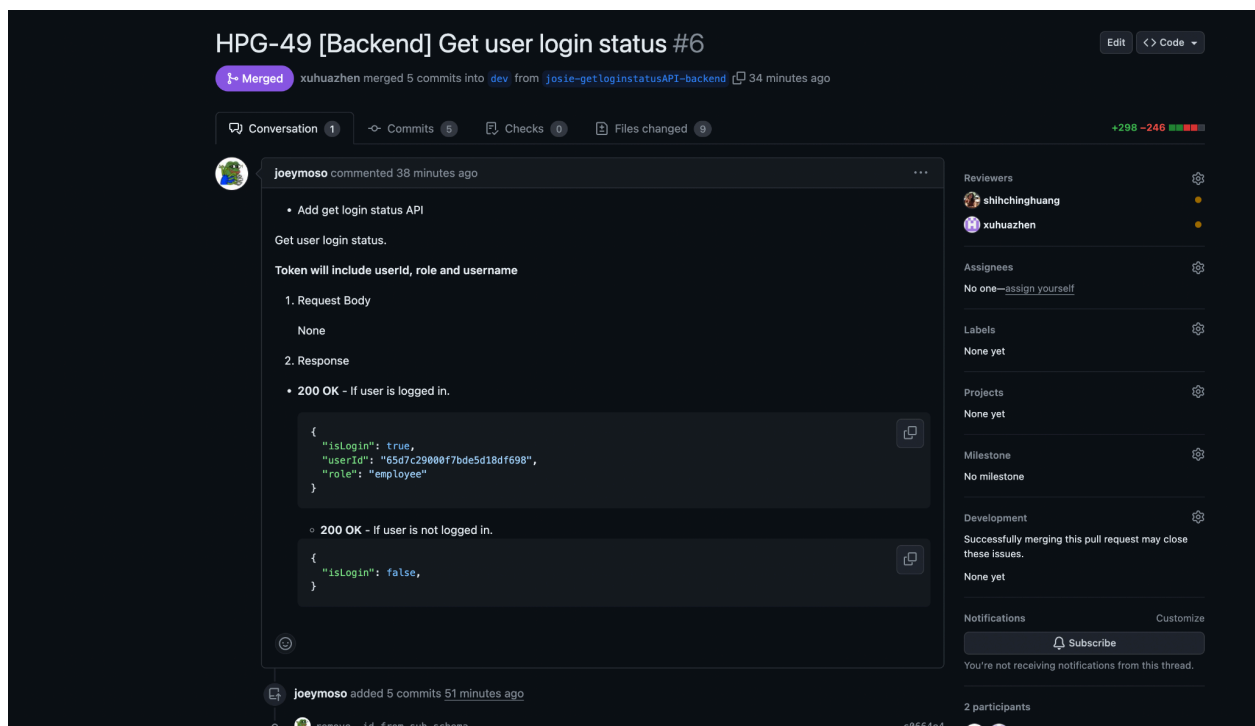
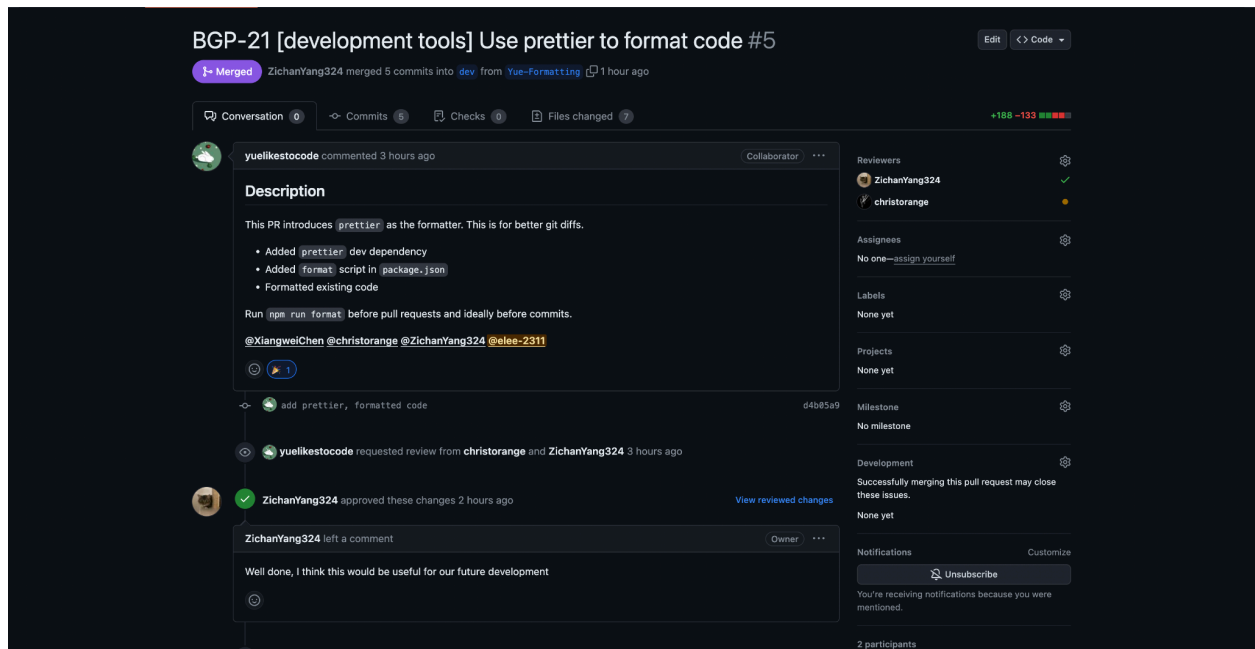
Notifications: Customize

Subscribe

You're not receiving notifications from this thread.

1 participant

ChuqiZeng



Once the PR is ready, your code will be undergoing CI/CD checks (linters, security checks, etc.). In most teams, only certain people can merge the code. **In Training, it is recommended that only the Team Lead has permission to merge or approve changes.** This can be configured in the repository's settings.

Code Review

Once the code has passed all CI/CD checks, you can request a team member or your team lead for a code review. In real work settings, code reviews often happen asynchronously (we don't want to block our team members from their current tasks with meetings). Team members may leave comments on the PR and block merging if there are issues with the code. Many teams will perform code reviews differently, but the most common points of a code review are as follows:

- Over-Engineering: is the approach and logic more complex than it needs to be?
- Under-Engineering: is the approach and logic hitting all of the story's AC?
- Semantic: are variables and functions appropriately named? Can another team member look at the name definition and understand what the variable is or what the function is doing?

Once a review has been conducted, those with write access may approve and merge the PR.

